Working with the LLM API

Hands-on Setup & First API Calls

Day 1 - Session 2

KavinSchool

Session 2: What We'll Cover

1. API accounts, keys, billing & rate limits
2. Our **provider-agnostic** setup (OpenAI / Gemini / Claude)
3. First API call: `chat.completions.create()`
4. Anatomy of a request and response
5. **Lab 1.1:** "Hello LLM"
6. **Lab 1.2:** Build a CLI Q&A script

API Keys & Setup

Your Gateway to LLM Power

KavinSchool

API Key Setup

Your `.env` file controls everything:

```
# Which provider to use: openai | gemini | claude
LLM_PROVIDER=openai

# Set the key for your chosen provider
OPENAI_API_KEY=sk-your-key-here
GEMINI_API_KEY=your-gemini-key-here
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Security rules:

- Never hardcode API keys in source code
- Never commit `.env` files to Git (it's in `.gitignore`)
- Never share keys in chat, email, or screenshots
- Rotate keys if accidentally exposed

Billing & Rate Limits

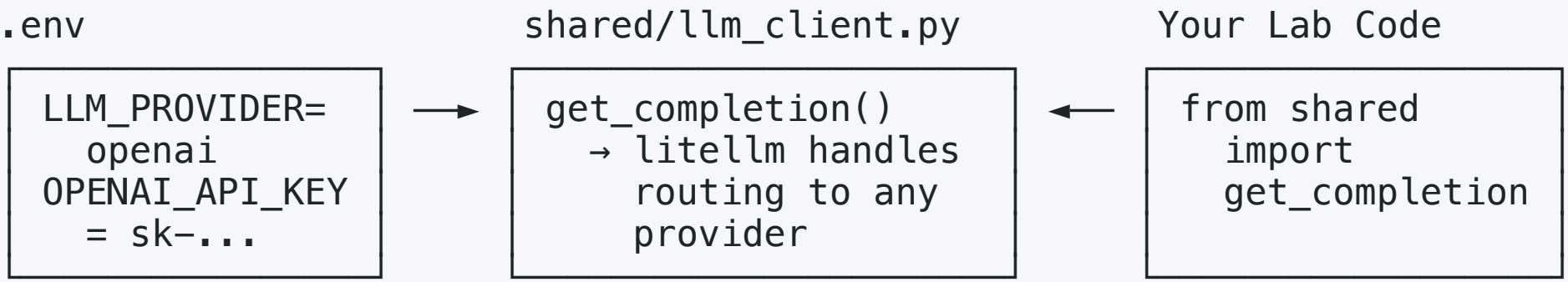
Provider	Pricing Model	Rate Limits (approx.)
OpenAI	Per-token (input + output)	500 RPM, 200K TPM
Gemini	Per-token (free tier available)	60 RPM, 4M TPM
Claude	Per-token (input + output)	50 RPM, 40K TPM

Typical costs for today's labs:

Model	Input (1K tokens)	Output (1K tokens)	Lab 1.1 est.
GPT-4o-mini	$0.00015	$0.0006	~$0.01
GPT-4o	$0.0025	$0.01	~$0.08
Gemini Flash	Free (up to limits)	Free	$0.00

Tip: Use the "mini" tier for experimentation, "default" for quality comparisons.

Our Provider-Agnostic Architecture



One codebase. Any provider. Change one line.

```
from shared import get_completion, show_config

show_config()  # verify your setup
```

Environment Check

Run this now:

```
python3 -c "
import sys; sys.path.insert(0, '.')
from shared.llm_client import show_config
show_config()
"
```

Expected output:

```
Provider:      openai
Default model: gpt-4o
Mini model:    gpt-4o-mini
API key:       OPENAI_API_KEY = ...abcd
```

> If you see `⚠️ NOT set`, check your `.env` file.

Your First API Call

From Zero to LLM in 5 Lines

KavinSchool

The Simplest Possible Call

```
from shared import get_completion

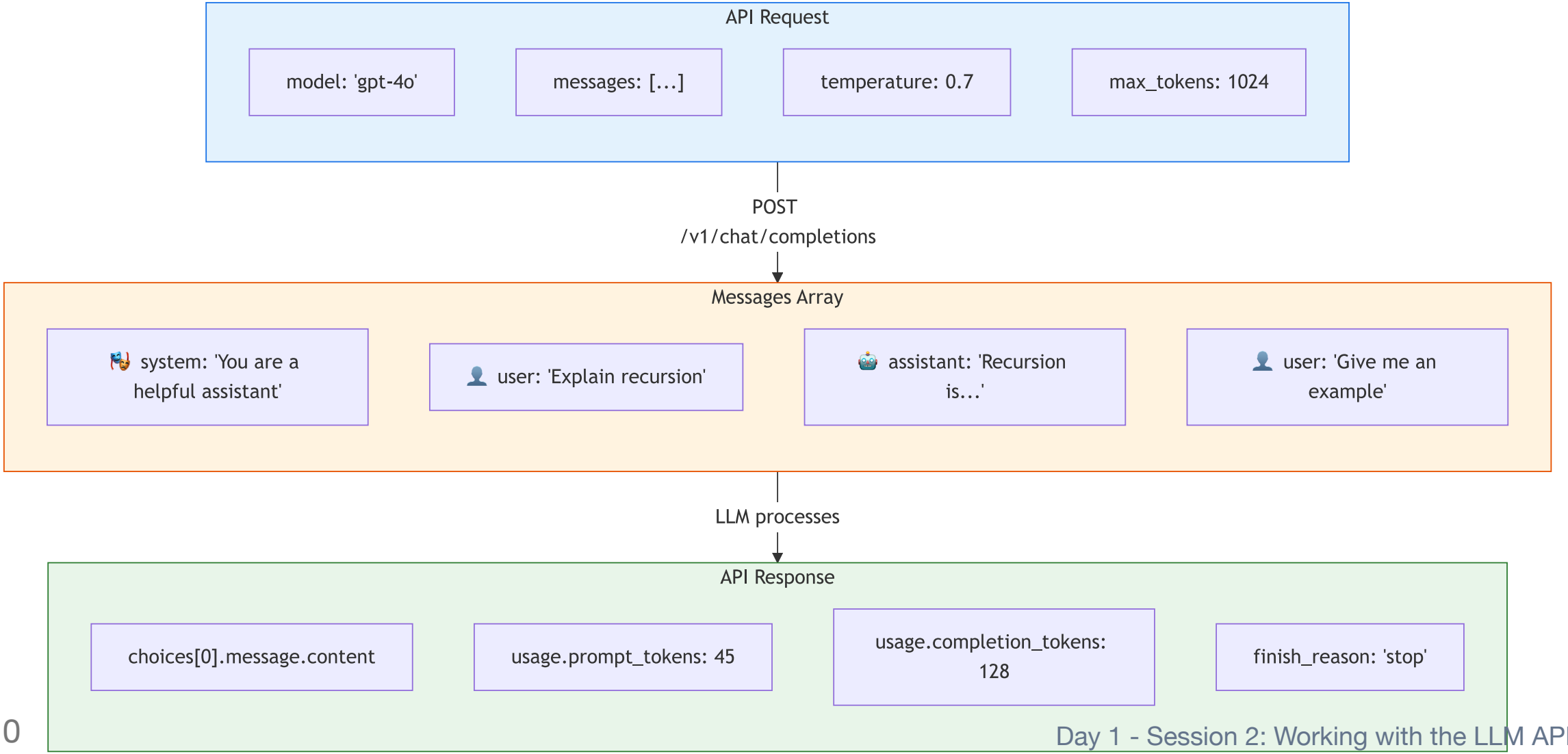
response = get_completion(
    messages=[
        {"role": "user", "content": "What is Python in one sentence?"}
    ]
)
print(response)
```

Output:

```
Python is a versatile, high-level programming language known for its
readable syntax and wide use in web development, data science, AI,
and automation.
```

That's it. **5 lines** to talk to the most powerful AI models on the planet.

Anatomy of a Request



The Messages Array: Conversation History

Messages use **roles** to structure the conversation:

Role	Purpose	Example
`system`	Sets behavior, personality, constraints	"You are a senior Python developer. Be concise."
`user`	The human's input	"How do I read a CSV file?"
`assistant`	Previous AI responses (for multi-turn)	"Use pandas: `pd.read_csv('file.csv')`"

```
messages = [
    {"role": "system", "content": "You are a helpful Python tutor. "
                                  "Explain concepts simply."},
    {"role": "user", "content": "What is a list comprehension?"},
]
```

> The system message shapes every response the model gives. Invest time crafting it.

Key Parameters

```
response = get_completion(
    messages=[...],
    tier="default",      # "default" = capable, "mini" = fast/cheap
    temperature=0.7,     # 0.0 = deterministic, 2.0 = creative chaos
    max_tokens=1024,     # max response length
)
```

Temperature: The Creativity Dial

Temperature	Behavior	Use Case
0.0	Always picks the most likely token	Code generation, data extraction
0.3	Mostly predictable, slight variation	Technical writing, summaries
0.7	Balanced (default)	General chat, explanations
1.0+	More random, creative	Brainstorming, creative writing

Anatomy of a Response

```python
from shared import get_completion_full

response = get_completion_full(
    messages=[{"role": "user", "content": "Say hello"}]
)

# The generated text
print(response.choices[0].message.content)  # "Hello! How can I help?"

# Token usage (this is what you pay for)
print(response.usage.prompt_tokens)      # 9
print(response.usage.completion_tokens)  # 6
print(response.usage.total_tokens)       # 15

# Why did it stop?
print(response.choices[0].finish_reason)  # "stop" | "length" | "content_filter"
```

Anatomy of a Response

finish_reason	Meaning
`stop`	Model completed its response naturally
`length`	Hit `max_tokens` limit, response truncated
`content_filter`	Content blocked by safety filters

Multi-Turn Conversations

LLMs are **stateless**. They don't remember previous calls. You must send the full conversation each time:

```
conversation = [
    {"role": "system", "content": "You are a Python tutor."},
    {"role": "user", "content": "What is a decorator?"},
]

# First turn
reply1 = get_completion(messages=conversation)
conversation.append({"role": "assistant", "content": reply1})

# Second turn: model sees the full history
conversation.append({"role": "user", "content": "Show me an example"})
reply2 = get_completion(messages=conversation)
```

> **Warning:** Each turn sends ALL previous messages → token costs grow with conversation length.

Common Mistakes to Avoid

Mistake	Problem	Fix
No system message	Model gives generic responses	Always set a system message
Temperature too high for code	Syntax errors, hallucinations	Use `temperature=0.0` for code
Ignoring `finish_reason`	Truncated output	Check for `"length"` and increase `max_tokens`
Sending the whole codebase	Exceeds context window, high cost	Send only relevant code
Hardcoding API keys	Security breach	Use `.env` + `python-dotenv`

Session 2: Key Takeaways

1. **API keys are secrets.** `.env` file, never commit, rotate if exposed
2. **The system message shapes everything.** Invest time crafting it
3. **Temperature controls creativity.** 0.0 for code, 0.7 for general use
4. **LLMs are stateless.** You must send full conversation history each turn
5. **Token costs add up.** Monitor usage, use mini models for experimentation

> Next: Session 3, Model Capabilities, Limitations & Input Preparation

Questions?

Session 2 Complete. Lunch Break!

KavinSchool